Proof of Concept: Agentic AI Development Workflow for the BTL Semantic Layer

1. Summary

This proof of concept demonstrates a lightweight, file-based agentic AI system for automating dbt data product development on the BTL (Business Translation Layer) Semantic Layer codebase.

The system consists of two types of reusable prompt files—**Foundational** and **Development**—supported by a structured context store (`.agent_context/`) that serves as the agent's knowledge base. Together, these components enable AI agents to build dbt models, schema definitions, and source configurations that conform to established team patterns and standards—without exceeding the AI model's context window.

The POC uses **Middle Market Americas (MMA)** as the first data product built through this system.

Key Outcomes

- AI agents can produce production-quality dbt code by reading curated context rather than scanning the full codebase (~1,900 models)
- Context is partitioned into bounded reference files, each scoped to a specific concern, keeping any single agent session well within window limits
- The system is product-agnostic — the same prompts and context structure apply to any new data product (cyber, gmr, claim, etc.)

---

2. Problem Statement

The core challenge can be stated as follows:

"How can a developer remain in full control of an outcome while reducing, or even removing, any remaining friction associated with the use of generative AI?"

Indeed, agentic coding assistants, such as GitHub Copilot and Claude Code, are already designed to reduce friction by integrating directly into the development experience and performing actual coding tasks with tremendous speed and efficiency. But how can we enable the AI to go beyond what they are able to do broadly by providing the detailed context in order to take advantage of their narrow use case? How do we keep a skilled engineer in the loop to guide the AI and ensure that key decisions align to the goals of the team at every stage of the development process for every data product delivered to the business customer?

Today, even with the help of an AI coding partner, this workflow is manual, knowledge-intensive, and difficult to scale as each new data product requires a skilled engineer who understands the full architecture, conventions, and governance requirements accumulated over years of development.

The BTL Semantic Layer team maintains a dbt codebase comprising three projects, ~1,900 SQL models, a 4,500-line shared metrics macro, and strict naming, configuration, and governance standards. Building a new data product today requires:

1. Understanding the architecture (pipeline ↔ semantic flow, LPL/SPL catalog separation, cross-project source wiring)
2. Knowing the naming conventions, model templates, config patterns, and testing standards
3. Reading a product specification (typically an Excel/CSV document with column mappings and business logic)
4. Writing pipeline models, semantic models, schema YAML, source YAML, and `dbt_project.yml` entries that comply with all standards
5. Validating the output against checklists before submission

This process is manual, knowledge-intensive, and error-prone. Onboarding a new team member (human or AI) requires absorbing context that is scattered across the codebase with no single reference point.

This proof of concept seeks to demonstrate that by providing a mechanism to centrally manage and integrate the knowledge, goals, and agenda of our team and that of the enterprise, agentic AI can automate much of the workflow while maintaining full control of the outcome. By and workflow necessary for a successfully delivered data product can make the integration as well as to inte enable **agentic AI can drive this workflow end-to-end** — from ingesting a product specification to producing deployable, standards-compliant dbt code — given the right context infrastructure. The static file approach used here is a pragmatic choice given current environmental constraints; a strategic implementation would replace these components with vector stores, MCP tools, and orchestration frameworks.

AI agents cannot simply be pointed at the full codebase because:

- The codebase is too large to fit in a single context window (~1,900 models, 43K source columns)
- Patterns and conventions are implicit — embedded in existing code, not documented
- Without curated context, agents produce code that compiles but violates team standards
- The development workflow spans multiple discrete steps (intake, mapping, build, validate) that must be sequenced correctly

---

3. High-Level Solution Design

3.1 Architecture

The solution has three layers:

```
ORCHESTRATION LAYER
.github/prompts/

  Foundational Prompts         Development Prompts
  (refresh context)           (build product code)
```

```
  writes                        reads + writes
        ↓                             ↓

CONTEXT STORE
.agent_context/
├── L1_ARCHITECTURE.md        (architecture overview)
├── L2/                       (patterns & standards)
├── L3/                       (product specs)
├── schemas/                  (source index + metadata CSVs)
├── governance/               (descriptions & test index)
└── requirements/{product}/   (input specs from business)

reads (via source())
        ↓

dbt CODEBASE
BTL_SEMANTIC_LAYER/
├── pipeline/           (940 models — views, raw + intermediate)
├── semantic/           (965 models — tables, business-facing)
└── shared_governance/  (macros, tests, descriptions)
```

3.2 Mapping to Standard Agentic AI Components

In a strategic agentic AI platform, context is typically managed by specialized infrastructure components. This POC implements equivalent functionality using files and prompts:

- **Knowledge Base / Vector Store (RAG)**
  Purpose: Store and retrieve relevant context on demand without loading everything
  POC implementation: `.agent_context/` — curated, pre-indexed reference files organized by concern. Each prompt declares exactly which files to read.
  Files: `L1`, `L2/*`, `schemas/source_index.md`, `governance/*`

- **System Instructions**
  Purpose: Persistent instructions that shape agent behavior for a task
  POC implementation: `.github/prompts/*_prompt.md` — each prompt file is a self-contained system instruction for a discrete operation
  Files: Foundational + Development prompts

- **Tool-Based Retrieval**
  Purpose: Fetch specific data on demand rather than pre-loading
  POC implementation: Terminal commands (`Import-Csv`, `Get-Content`) to query schema CSVs and read Copilot-ignored files
  Files: `schemas/*.csv`, `requirements/*/*.csv`

- **Memory / State Store**
  Purpose: Persist outputs across sessions so one agent's output feeds the next
  POC implementation: L3 product specs — written by one prompt, consumed by the next in the development workflow
  Files: `L3/{product}.md`

- **Context Manifest**
  Purpose: Declare what an agent needs to read, with budget constraints
  POC implementation: Prompt headers listing input files, approximate sizes, and total read

budget
Files: Prompt `## Context Budget` sections

3.3 Context Window Management

Each prompt is designed to stay within the AI model's context window by:

1. **Declaring a context budget** — listing input files and their sizes so the total read load is predictable
2. **Reading only what's needed** — a pipeline build prompt reads model templates and naming conventions, not the governance index
3. **Producing bounded outputs** — each prompt writes a defined artifact (spec, SQL file, YAML) rather than attempting the full product build
4. **Using tool-based retrieval for large data** — Schema CSVs (4.4 MB LPL, 334 KB SPL) are never loaded whole; agents query them via terminal for specific tables

---

4. Context Store — Reference Files

4.1 Architecture & Patterns (always available, read selectively)

- `L1_ARCHITECTURE.md` — **18.6 KB**
  Three-project architecture, data flow, catalog model, cross-project wiring
  When referenced: Foundational refresh; development intake
- `L2/L2_01_QUICK_REFERENCE.md` — **3.6 KB**
  Cheat sheet: project names, materializations, tags, LPL/SPL product lists
  When referenced: Every development prompt
- `L2/L2_02_NAMING_CONVENTIONS.md` — **4.2 KB**
  File, alias, and column naming rules
  When referenced: Pipeline + semantic build prompts
- `L2/L2_03_MODEL_TEMPLATES.md` — **9.4 KB**
  Six model templates (A–F) with skeleton SQL
  When referenced: Pipeline + semantic build prompts
- `L2/L2_04_SCHEMA_YML_PATTERNS.md` — **3.2 KB**
  `_schema.yml` conventions and examples
  When referenced: Pipeline + semantic build prompts
- `L2/L2_05_SOURCE_YML_PATTERNS.md` — **5.0 KB**
  `_source.yml` cross-project wiring patterns
  When referenced: Semantic build prompt
- `L2/L2_06_MACRO_REFERENCE.md` — **6.9 KB**
  All macro signatures; `getdynamicmeasures` documentation
  When referenced: Pipeline + semantic build (when measures are needed)

- `L2/L2_07_GENERIC_TESTS.md` — **3.4 KB**
  Three dimension quality tests with usage examples
  When referenced: Validation prompt
- `L2/L2_08_CONFIG_PATTERNS.md` — **7.9 KB**
  Full `dbt_project.yml` config matrix
  When referenced: Config integration prompt
- `L2/L2_09_PRODUCT_CATALOG.md` — **5.2 KB**
  Complete product inventory with model counts
  When referenced: Foundational refresh; development intake
- `L2/L2_10_DECISION_TREE.md` — **6.6 KB**
  Decision trees + pre-submission checklists
  When referenced: Validation prompt

4.2 Indexes (generated by foundational prompts)

- `schemas/source_index.md` — **14.3 KB**
  Cross-reference: 922 catalog tables ↔ 42 source YAML definitions, catalog summaries, product-to-catalog mapping
  When referenced: Source mapping prompt
- `governance/descriptions_index.md` — **8.5 KB**
  38 doc blocks, 7 macros, 3 generic tests — full shared_governance inventory
  When referenced: Development prompts (as needed)

4.3 Data Files (queried via terminal, not loaded into context)

- `schemas/axiom_etl_prd_lpl_btl_src.csv` — **4,409 KB**
  900 tables, 40,185 columns — LPL catalog metadata
  When referenced: Source mapping (filtered lookup)
- `schemas/axiom_etl_prd_spl_btl_src.csv` — **334 KB**
  22 tables, 2,918 columns — SPL catalog metadata
  When referenced: Source mapping (filtered lookup)
- `requirements/{product}/*.csv` — **Varies**
  Product requirement specs (columns, filters, measures)
  When referenced: Development intake prompt

4.4 Product Specs (generated by development prompts, consumed by subsequent steps)

- `L3/{product}.md` — **~20–40 KB**
  Complete product spec: requirements analysis, source mappings, proposed model design, column matrix
  When referenced: Pipeline build, semantic build, config, validation

---

5. Prompt Inventory

5.1 Foundational Prompts

These prompts refresh the context store. Run them when the codebase changes, governance docs are updated, new products are added, or schema metadata is refreshed.

- **F1: Architecture Scan**
  Reads: Full codebase structure, `dbt_project.yml` files, `packages.yml`
  Produces: Updated `L1_ARCHITECTURE.md`
  Trigger: Major structural changes, new projects added
- **F2: Patterns Refresh**
  Reads: `dbt_project.yml` configs, existing models (sampling), naming patterns in source code
  Produces: Updated `L2/` files (10 files)
  Trigger: New patterns emerge, config changes, macro updates
- **F3: Source Index Refresh**
  Reads: Schema CSVs (via terminal), all `_source.yml` files
  Produces: Updated `schemas/source_index.md`
  Trigger: User drops new schema CSVs, source YAMLs added/changed
- **F4: Governance Index Refresh**
  Reads: `shared_governance/macros/descriptions.md`, `_schema.yml` files for `doc()` refs, generic test files
  Produces: Updated `governance/descriptions_index.md`
  Trigger: Doc blocks added/changed, new tests created
- **F5: Product Catalog Refresh**
  Reads: All product folders in `pipeline/models/` and `semantic/models/`, model counts
  Produces: Updated `L2/L2_09_PRODUCT_CATALOG.md`
  Trigger: New products deployed, products retired

5.2 Development Prompts

These prompts execute sequentially to build a data product. Each reads defined context files and produces an artifact consumed by the next step.

- **D1: Product Intake**
  Reads: Requirement CSVs (via terminal), `L2_01` (quick ref), `L2_09` (product catalog)
  Produces: `L3/{product}.md` — requirements analysis, column inventory, filter logic, source table candidates
  Context budget: ~35 KB read
- **D2: Source Mapping**
  Reads: `L3/{product}.md`, `schemas/source_index.md`, schema CSVs (filtered via terminal)
  Produces: Updated `L3/{product}.md` — source mappings validated, column matrix

with source `table.column`
Context budget: ~50 KB read

- **D3: Pipeline Build**
  Reads: `L3/{product}.md` (source mapping section), `L2_02` (naming), `L2_03` (templates), `L2_04` (schema YML)
  Produces: Pipeline `.sql` models + `_schema.yml`
  Context budget: ~40 KB read

- **D4: Semantic Build**
  Reads: `L3/{product}.md`, `L2_02`, `L2_03`, `L2_04`, `L2_05` (source patterns), `L2_06` (macros, if measures needed)
  Produces: Semantic `.sql` models + `_schema.yml` + `_source.yml`
  Context budget: ~45 KB read

- **D5: Config Integration**
  Reads: `L3/{product}.md` (product metadata), `L2_08` (config patterns)
  Produces: `dbt_project.yml` additions for both pipeline and semantic
  Context budget: ~30 KB read

- **D6: Validation**
  Reads: All generated files, `L2_10` (checklists), `L2_02` (naming), `L2_07` (generic tests)
  Produces: Pass/fail report with issues to fix
  Context budget: ~30 KB read

5.3 Workflow Sequence

Foundational (as needed)
F1: Architecture Scan
F2: Patterns Refresh
F3: Source Index Refresh
F4: Governance Refresh
F5: Product Catalog Refresh

Development (per product)
D1: Product Intake
D2: Source Mapping
D3: Pipeline Build
D4: Semantic Build
D5: Config Integration
D6: Validation

Each `D` prompt is a **separate agent session**. The L3 spec file (`D1`/`D2` output) is the persistent state that chains the sessions together — serving as the system's memory store.

---

6. Current Status

Built

- `.agent_context/` folder structure — ✅
  Excluded from dbt via `.dbtignore`
- L1 Architecture doc — ✅
  18.6 KB
- L2 Patterns & Standards (10 files) — ✅
  55.4 KB total
- Schema index + source cross-reference — ✅
  14.3 KB
- Governance Index — ✅
  8.5 KB
- Schema CSVs (user-provided) — ✅
  LPL: 4.4 MB, SPL: 334 KB
- MMA requirement CSVs (user-provided) — ✅
  3 CSVs + Excel source

To Build

- Foundational prompt files (F1–F5) — ⬜
  Define in `.github/prompts/`
- Development prompt files (D1–D6) — ⬜
  Define in `.github/prompts/`
- L3 MMA spec (via D1 + D2) — ⬜
  First product to validate the system
- MMA code output (via D3–D6) — ⬜
  Pipeline + semantic models
- Temp file cleanup — ⬜
  Remove `_temp_*` files from `schemas/`

---

7. Risks & Limitations

- **Context window exceeded during large product intake**
  Mitigation: Context budget in prompt headers; split D1 if requirement CSV > 100 KB
- **Schema CSVs are Copilot-ignored** (`.csv` files can't be read via `read_file`)
  Mitigation: Prompts instruct agents to use terminal `Import-Csv` / `Get-Content` for CSV access
- **L2 files become stale as codebase evolves**
  Mitigation: Foundational prompts (F2–F5) provide a repeatable refresh mechanism

- `getdynamicmeasures` **macro is 4,536 lines — too large to load fully**
  Mitigation: `L2_06_MACRO_REFERENCE.md` documents the signature and top measures; agents grep for specific measures on demand
- **Single-agent sessions can't share live state**
  Mitigation: L3 spec file serves as persistent memory between sessions
- **POC is model-specific (Claude/Copilot)**
  Mitigation: Prompt files are plain markdown — portable to other agent frameworks

---

8. What Comes Next

8.1 Strategic Agentic AI

"Beyond this POC", a strategic implementation might replace file-based components with:

- **Vector store** for the knowledge base — enabling semantic retrieval instead of explicit file lists in prompts
- **MCP (Model Context Protocol) tools** for schema lookup — replacing terminal CSV queries with structured tool calls
- **Orchestration framework** (e.g., LangGraph, AutoGen) — automating the D1–D6 sequence with conditional routing
- **CI/CD integration** — foundational prompts triggered by PR merges; development prompts triggered by ticket creation
- **Multi-agent collaboration** — parallel pipeline and semantic builds with a coordinator agent

The file-based approach in this POC is intentionally simple — it validates the decomposition and context budgets before investing in infrastructure.

8.2 Orchestration

In this POC, the **human acts as the orchestrator** — manually invoking each prompt (D1, then D2, etc.) in separate Copilot chat sessions. The L3 spec file on disk is what chains the sessions together: each prompt produces a defined artifact that the next prompt consumes.

GitHub Copilot's current agent mode is single-threaded — one agent, one session, one task at a time. There is no built-in mechanism to spawn multiple agents in parallel, chain sessions programmatically (D1 finishes → auto-trigger D2), or have an orchestrator agent delegate subtasks to worker agents.

Current Capabilities

- **Copilot Agent Mode** — ✅ Available
  Single agent executes one prompt per session
- **Copilot custom instructions** (`.github/copilot-instructions.md`) — ✅ Available
  Persistent system instructions across all sessions

- **Copilot prompt files** (`.github/prompts/`) — ✅ Available
  Reusable task-specific prompts — **what this POC uses**
- **MCP tools** — 🔄 Rolling out
  Could provide structured schema lookups instead of terminal CSV queries
- **Copilot Coding Agent** (async, PR-based) — 🔵 Preview
  Runs autonomously on a GitHub issue, creates a PR — could run D3–D6 unattended
- **Multi-agent orchestration** — ❌ Not in Copilot
  Would require external frameworks

Path to Automated Orchestration

To move beyond human-as-orchestrator, the options are:

1. **LangGraph / AutoGen / CrewAI** — Python-based multi-agent frameworks that can spawn parallel agents, each with their own context window and tool access. The foundational and development prompts translate directly into agent task definitions.
2. **GitHub Actions + Copilot Coding Agent** — Trigger the coding agent via GitHub issue creation within a CI workflow. A GitHub Action could create an issue with the D1 prompt content, the coding agent produces the L3 spec in a PR, and a subsequent action triggers D3–D6 as follow-on issues.
3. **Custom MCP server** — Expose the context store (`.agent_context/`) and dbt operations (compile, run, test) as structured tools that any agent framework can call, replacing the terminal-based file reads used in this POC.

Why the POC Design Still Applies

The prompt decomposition (F1–F5, D1–D6) is intentionally framework-agnostic. Each prompt:

- Declares its inputs and outputs explicitly
- Stays within context window budgets
- Produces a defined artifact on disk

This means the prompts work equally well whether invoked by a human, a Copilot chat session, a LangGraph node, or a GitHub Actions workflow. Getting the decomposition right now avoids redesign when orchestration infrastructure becomes available.

VS Code Extension

A custom VS Code extension could provide a purpose-built orchestration interface for human-driven agent workflows, transforming the current manual process of finding and invoking prompt files into a structured, guided experience.

Such an extension could offer:

- **Workflow panel** — A sidebar view showing the full prompt sequence (F1–F5, D1–D6) with status indicators (not started, in progress, completed) for the current product. The human orchestrator sees at a glance where they are in the workflow and what comes next.
- **One-click prompt invocation** — Each step in the workflow panel launches a Copilot agent session pre-loaded with the correct prompt file and relevant context file references. The

orchestrator doesn't need to remember which prompt to use or which context files it requires — the extension handles the wiring.

- **Context budget visibility** — Before launching a prompt, the extension calculates the total size of input files declared in the prompt's context budget and displays whether it fits within the model's context window. This prevents failed sessions due to overloaded context.
- **Artifact tracking** — As each prompt produces its output artifact (L3 spec, SQL files, YAML files), the extension detects the new files and advances the workflow status. If a required input artifact for the next step is missing, the extension surfaces the dependency.
- **Product management** — A dropdown or configuration panel to select the active product (MMA, cyber, gmr, etc.), which automatically adjusts file paths in the prompt templates (e.g., `requirements/{product}/` and `L3/{product}.md`).
- **Foundational prompt scheduling** — Visual indicators showing when foundational context files are stale (e.g., `source_index.md` is older than the latest `_source.yml` change), prompting the orchestrator to run the relevant foundational refresh before proceeding with development prompts.

This approach keeps the human in the loop as the decision-maker while removing the friction of navigating prompt files, remembering sequencing, and manually tracking progress — bridging the gap between the fully manual orchestration in this POC and the fully automated multi-agent frameworks described above.

8.3 Development Approach Modes

The current POC follows a **single approach**; the agent reads the context store and the product specification (STM), then proceeds through D1–D6 autonomously, making design decisions based on patterns it finds in the L2 reference files and existing codebase examples. This works — but every product's specification is different, and the agent's design choices during D1–D3 (intake, source mapping, pipeline build) are where the most consequential architectural decisions happen. A wrong assumption here compounds through every downstream step.